Logging will always have a small impact on the performance of your Rails app, particularly when logging to disk. Additionally, there are a few subtleties:

Using the :debug level will have a greater performance penalty than :fatal, as a far greater number of strings are being evaluated and written to the log output (e.g. disk).

Another potential pitfall is too many calls to Logger in your code:

```
logger.debug "Person attributes hash: #{@person.attributes.inspect}"
logger.debug "Person attributes hash: #{@person.attributes.inspect}"
```

In the above example, there will be a performance impact even if the allowed output level doesn't include debug. The reason is that Ruby has to evaluate these strings, which includes instantiating the somewhat heavy String object and interpolating the variables. Therefore, it's recommended to pass blocks to the logger methods, as these are only evaluated if the output level is the same as — or included in — the allowed level (i.e. lazy loading). The same code rewritten would be:

```
logger.debug {"Person attributes hash: #{@person.attributes.inspect}"}
logger.debug {"Person attributes hash: #{@person.attributes.inspect}"}
```

The contents of the block, and therefore the string interpolation, are only evaluated if debug is enabled. This performance savings are only really noticeable with large amounts of logging, but it's a good practice to employ.

3 Debugging with the byebug gem